1. Project Introduction

Project Title

Security Testing of OWASP Juice Shop Web Application

1.1 Introduction

In the modern digital era, web applications are widely used for handling sensitive user data, financial transactions, and business operations. As web applications grow in complexity, they also become more vulnerable to security threats such as injection attacks, cross-site scripting, authentication bypass, and session hijacking.

To address these challenges, security testing plays a critical role in identifying vulnerabilities before they can be exploited by malicious users.

The OWASP Juice Shop Web Application is a deliberately insecure web application developed by the OWASP Foundation. It is widely used for security training, ethical hacking practice, and penetration testing exercises. The application contains a wide range of real-world vulnerabilities that simulate actual production security issues.

This project focuses on performing **comprehensive security testing of OWASP Juice Shop using both manual testing techniques and an automation testing framework built using Selenium WebDriver and TestNG**.

The automation framework follows the **Page Object Model (POM) design pattern**, which ensures modularity, maintainability, and scalability of test scripts. The framework is capable of executing multiple security-related test scenarios such as SQL Injection, XSS attacks, input validation testing, authentication validation, and session management verification.

The combination of manual and automated testing ensures maximum coverage of security vulnerabilities and provides a structured approach to identifying, reporting, and analyzing security risks.

1.2 Background of the Application

OWASP Juice Shop is one of the most popular insecure web applications used globally for learning application security. It is intentionally designed with vulnerabilities that reflect real-world security flaws found in modern web applications.

It includes vulnerabilities such as:

- SQL Injection
- Cross-Site Scripting (XSS)
- Broken Authentication
- Sensitive Data Exposure
- Security Misconfiguration
- Improper Input Validation

This makes it an ideal platform for practicing security testing techniques and automation framework development.

1.3 Problem Statement

Most web applications today are vulnerable due to improper input validation, weak authentication mechanisms, and insecure coding practices. Manual testing alone is not sufficient to identify all vulnerabilities efficiently, especially when regression testing is required frequently.

There is a need for an **automated security testing framework** that can:

- Execute repetitive security test cases
- Validate multiple attack scenarios efficiently
- Reduce manual effort and human error
- Provide structured reporting and evidence collection

1.4 Purpose of the Project

The main purpose of this project is to design and implement a structured security testing approach for a vulnerable web application using automation tools.

This includes:

- Identifying security vulnerabilities in OWASP Juice Shop
- Validating application behavior against malicious inputs
- Automating security test cases using Selenium WebDriver
- Generating structured reports for analysis
- Documenting vulnerabilities and mitigation strategies

1.5 Objectives of the Project

The key objectives of this project are:

- To perform security testing on OWASP Juice Shop application
- To identify vulnerabilities such as SQL Injection and XSS
- To design structured test scenarios and test cases
- To implement an automation framework using Selenium and TestNG
- To execute automated regression security tests
- To validate authentication, input handling, and session management
- To generate execution reports and defect logs
- To improve understanding of secure software development practices

---

1.6 Scope of the Project

In Scope:

- Security testing of OWASP Juice Shop web application
- Automation of security test cases using Selenium WebDriver
- Implementation of Page Object Model (POM) framework
- Execution of:
    - SQL Injection test cases
    - Cross-Site Scripting (XSS) test cases
    - Input validation testing
    - Authentication and authorization testing
    - Session security testing
    - Error handling validation
- Generation of test reports and screenshots
- Documentation of security vulnerabilities and mitigation strategies

Out of Scope:

- Source code modification of the application
- Network-level penetration testing
- Infrastructure or server-side security testing beyond application layer

---

1.7 Methodology Followed

The following methodology is used in this project:

1. Requirement analysis of security testing scope
2. Identification of security test scenarios
3. Design of test cases for vulnerabilities
4. Development of automation framework using Selenium
5. Execution of test cases using TestNG
6. Capturing execution results and screenshots
7. Analysis of failed test cases
8. Reporting of vulnerabilities and risks
9. Suggestion of mitigation strategies

1.8 Automation Framework Overview

The automation framework is developed using a structured architecture based on industry standards.

Technologies Used:

- Java (Programming Language)
- Selenium WebDriver (Automation Tool)
- TestNG (Testing Framework)
- Maven (Build Management Tool)
- Eclipse IDE

Framework Design Pattern:

- Page Object Model (POM)

Framework Structure:

- Base Layer:
  - BaseTest.java → Handles browser setup and teardown
  - DriverFactory.java → Manages WebDriver instances
- Page Layer:
  - LoginPage.java
  - HomePage.java
  - FeedbackPage.java
- Test Layer:
  - Authentication_Test.java
  - SQLInjection_Test.java
  - XSS_Test.java
- Utility Layer:
  - ConfigReader.java
  - ScreenshotUtil.java
  - ExcelReader.java
- Listener Layer:
  - TestListener.java (for reporting and logging)

Advantages of Framework:

- Easy maintenance of test scripts
- High reusability of code
- Scalable structure for future test cases
- Better reporting and debugging support

1.9 Expected Outcome

After completing this project:

- Security vulnerabilities will be successfully identified
- Automation framework will execute security test cases efficiently
- Test reports (HTML/XML) will be generated using TestNG
- Defects will be documented in bug reports
- Screenshots will provide execution evidence
- Mitigation strategies will be defined for each vulnerability

---

1.10 Conclusion

This project demonstrates a complete approach to security testing of a web application using both manual and automation techniques. It helps in understanding real-world security vulnerabilities and how they can be detected using structured testing practices.

The automation framework enhances efficiency, reduces manual effort, and ensures consistent validation of security test scenarios.
